

XRPL Validator — Daily Engineering Report • 2026-09-24

Generated 2026-09-25 00:20 EDT. Scope: XRPL work only — the git history of [xrpl-validator](#) and the XRPL Rust SDK, the live native-engine shadow, the reference rippled node, the XRP Ledger itself (amendments) and XRPL research.

Summary

- **11 commits** (xrpl-validator 11)
- **2 engine findings fixed** (F399–F400)
- Live shadow: 4,910/4,910 ledgers fully matched, 490,987 transactions, 0 engine receipts in the current soak
- Amendment **PermissionDelegationV1_1** gained majority at ledger 107,212,033 (2026-09-24 21:25 UTC)
- Amendment **BatchV1_1** held majority (since 2026-09-15 14:06 UTC); activation due ~2026-09-29 14:06 UTC
- Amendment **PermissionDelegationV1_1** held majority (since 2026-09-24 21:25 UTC); activation due ~2026-10-08 21:25 UTC

Live validator and native engine

Snapshot at generation time.

Measure	Value
Ledgers compared (current run)	4,910
Ledgers fully matched	4,910
Transactions applied natively	490,987
Result mismatches	0
Byte mismatches	0
Missing / extra keys	0 / 0
Batch inner-result mismatches	0
Engine receipts (current soak)	0
Live engine build running since	2026-09-24 18:58 EDT
Reference rippled	3.4.0, full, validated #107,218,509

Engine findings fixed

- **fix(ledger): F399 — two-asset AMM withdrawals check the reserve before each new trust line (soak #28 receipt)** (ea76860 , xrpl-validator) rippled's `AMMWithdraw::withdraw()` runs `sufficientReserve` for each asset right before its send, in every withdrawal mode (`AMMWithdraw.cpp` 765/785): a withdrawal that would create a trust line for an account owning two or more objects needs `base + increment x (OwnerCount + 1)` against the balance before the fee. The single-asset paths had this check (finding 156); the two-asset equal withdrawals (`tfWithdrawAll` and ...

- **fix(ledger): F400 — NFT IOU settlement grosses the transfer fee to nearest, as accountSend does (soak #28 receipt)** (a08b8ee , xrpl-validator) An NFTTokenAcceptOffer settles in an IOU with rippled's `pay()` = `accountSend`, whose third-party send grosses the sender's debit with `multiply(saAmount, transferRate)` (TokenHelpers.cpp:869): `STAmount multiply, Number{v1} * Number{v2}`, rounded to the NEAREST 16-digit value. The engine used `gross_in`, the payment engine's round-up `mulRatio` (correct for BookStep, not for `accountSend`).

Engineering changes

- **fix(node): warm restart scans the wallet count before ws-sync adjusts it** (956ec00 , xrpl-validator) Both cold startup branches scan the AccountRoot count; the warm branch did not, so the counter started at 0 and the first deleted AccountRoot wrapped it to $2^{64} - 1$ on the dashboard. `run_warm_resume` now scans it beside the hasher rebuild (no added restart time). Found by the final whole-branch review.
- **feat(node): resume_check — rehearse the warm-restart check on a kept state, read-only** (99b7e0d , xrpl-validator)
- **feat(node): warm restart — clean stop writes a resume ticket; XRPL_WARM_RESUME starts from the verified bookmark** (a2518d1 , xrpl-validator)
- **feat(node): warm restart — the resume sequence: ticket, bookmark, gap cap, root against the network** (c8b4ad6 , xrpl-validator)
- **feat(node): warm restart — ws-sync parks at a ledger boundary; every rollback rewinds meta:last_seq** (c058ccf , xrpl-validator)
- **feat(node): warm restart — a stop gate that parks ws-sync at a ledger boundary** (2838f58 , xrpl-validator)
- **feat(node): warm restart — resume ticket, meta:last_seq bookmark and the resume checks** (2d751ef , xrpl-validator)

Docs and operations

- **docs: warm restart design — resume from the verified ledger bookmark, cold fallback** (43fb741 , xrpl-validator)
- **docs: warm restart design — incremental syncer off first, 1,000-ledger gap cap, deploy verdict timeout** (88f4057 , xrpl-validator)

Notes

Hand-written notes on the day's XRPL work: research and operations.

- **Warm restart (merged, not yet deployed).** Every deployment has restarted the node from an empty store and re-downloaded the full ledger state (about 19.9 million objects), so it resumed 90–111 ledgers behind the network. The node can now resume from its own saved state instead: a clean stop parks the ledger-sync loop between ledgers and records the last verified ledger; on restart the node checks that its stored state names that ledger, rebuilds the state tree and requires its root to equal the network's account hash for that ledger before continuing. Any doubt falls back to the full resynchronisation. Earlier attempts to restart without resynchronising (April and

August) failed because they resumed from the wrong ledger; this design verifies the ledger first. A rehearsal on two real mainnet states kept from the 2026-09-23 deployments rebuilt both to the network's account hash (31–34 s each). The first deployment that carries this code still starts cold; the one after it is the first warm restart.

- **Soak #28 receipt and finding F399.** At ledger 107,210,679 an AMMWithdraw (withdraw all, from a BTC/xSPECTAR pool) came from an account holding 2.799986 XRP at an owner count of 9 with no trust line for either pool asset. rippled checks the reserve for each new trust line before paying each asset out, in every withdrawal mode; a new line at that owner count needs 3 XRP, so mainnet returned `tecINSUFFICIENT_RESERVE`. The engine's two-asset withdrawal paths (withdraw all, and withdrawal by LP-token amount) skipped that check and paid both assets out. Fixed (F399) with the mainnet transaction pinned as a regression vector; verified and merged; deployment awaits a decision.
- **Operations.** At 01:40–01:55 EDT a test build on the machine that runs the reference rippled node pushed it into memory pressure; its validated ledger fell about ten minutes behind and the live node paused until it recovered (no mismatches, no engine receipts). Full test batteries no longer run on that machine while the live node depends on it.
- **Second soak #28 receipt and finding F400.** At ledger 107,211,701 an NFTTokenAcceptOffer settled a buy offer priced 211,299.047746283 SOLO; the SOLO issuer charges a 0.01% transfer fee, so the buyer's debit is the price times 1.0001. rippled grosses that debit with its standard amount multiplication, rounded to the nearest 16-digit value (211,320.1776510576); the engine used the payment engine's round-up rule for the same product (...0577), leaving the buyer's SOLO balance one unit lower in the sixteenth digit. Earlier NFT sales with a transfer fee had exact products and could not show the difference. Fixed (F400) with the mainnet transaction pinned as a regression vector; verified and merged; deployment awaits a decision.
- **Deployment (F398–F400 and the warm restart).** Build a08b8ee replaced the live shadow build at 18:58 EDT after its gate (1,621 suite tests and 707 flow-engine tests passing) and both mainnet replay windows (60 and 98 ledgers, zero mismatches). This deployment still started from a full resynchronisation, as designed (the previous build could not leave a resume ticket). The health check matched 408 ledgers with no mismatches, and soak #29 began at 19:26 EDT (seed ledger 107,213,499). The receipt canary confirmed the comparison is live on the new build.
- **Build infrastructure.** Test gates now run on a dedicated build machine instead of the machine that hosts the reference rippled node, whose disk the test builds had been saturating. The gate completed in 10 minutes (previously about 15) with no effect on the reference node.

Amendment watch

Read from the XRP Ledger: the Amendments object as of the last ledger of the day, and the day's EnableAmendment pseudo-transactions.

Voting events:

- PermissionDelegationV1_1 gained majority — ledger 107,212,033, 2026-09-24 21:25 UTC

Amendment	Majority since	Activation due
BatchV1_1	2026-09-15 14:06 UTC	2026-09-29 14:06 UTC
PermissionDelegationV1_1	2026-09-24 21:25 UTC	2026-10-08 21:25 UTC

XRPL research

Automated research — candidate amendment ideas

Ideas generated and prior-art-checked by the XRPL Amendment Board, an automated research tool (read-only board: iceland.halcyon-names.io). They are exploration leads, not proposals.

- **Vault-Scoped Delegated Session Keys** (custody) Add a SessionKey ledger object (ltSESSION_KEY, owner-reserved, keylet::sessionKey(account, sequence)) created via a new SessionKeySet transaction carr
- **Vault Collateral Clawback Settlement** (lending) Add an lsfVaultClawbackable flag to the Vault SLE, settable only at VaultCreate when the vault's underlying asset issuer holds clawback rights (AMMClawbackable)
- **Credential-Discounted Vault Collateral Ratios** (lending) Add sfCreditCredentialType and sfMaxUncollateralizedBps SFields to LoanBroker/Vault creation, letting a vault owner opt in to accepting a specific Cre

Appendix — commits

Time	Repo	Commit	Subject
00:35	xrpl-validator	43fb741	docs: warm restart design — resume from the verified ledger bookmark, cold fallback
00:39	xrpl-validator	88f4057	docs: warm restart design — incremental syncer off first, 1,000-ledger gap cap, deploy verdict timeout
02:16	xrpl-validator	956ec00	fix(node): warm restart scans the wallet count before ws-sync adjusts it
02:16	xrpl-validator	99b7e0d	feat(node): resume_check — rehearse the warm-restart check on a kept state, read-only
02:16	xrpl-validator	a2518d1	feat(node): warm restart — clean stop writes a resume ticket; XRPL_WARM_RESUME starts from the verified bookmark
02:16	xrpl-validator	c8b4ad6	feat(node): warm restart — the resume sequence: ticket, bookmark, gap cap, root against the network
02:16	xrpl-validator	c058ccf	feat(node): warm restart — ws-sync parks at a ledger boundary; every rollback rewinds meta:last_seq
02:16	xrpl-validator	2838f58	feat(node): warm restart — a stop gate that parks ws-sync at a ledger boundary
02:16	xrpl-validator	2d751ef	feat(node): warm restart — resume ticket, meta:last_seq bookmark and the resume checks
16:11	xrpl-validator	ea76860	fix(ledger): F399 — two-asset AMM withdrawals check the reserve before each new trust line (soak #28 receipt)
17:15	xrpl-validator	a08b8ee	fix(ledger): F400 — NFT IOU settlement grosses the transfer fee to nearest, as accountSend does (soak #28 receipt)